



Data Structures and Algorithms

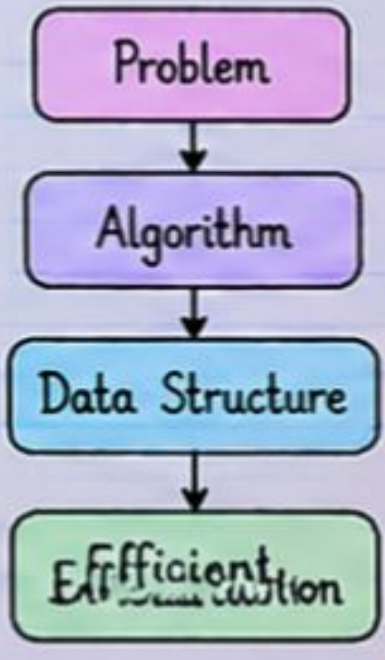
Handwritten Notes

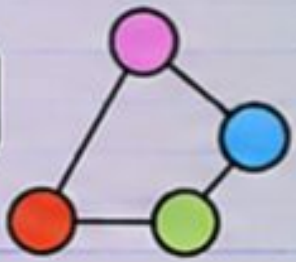
From Basics to Advanced

Topics Covered :

- Arrays
- Linked List
- Stack & Queue
- Trees
- Graphs
- Searching & Sorting
- Recursion
- Dynamic Programming
- Greedy Algorithm
- Backtracking
- Time & Space Complexity
- And More...







Roshani Kumari

Contents

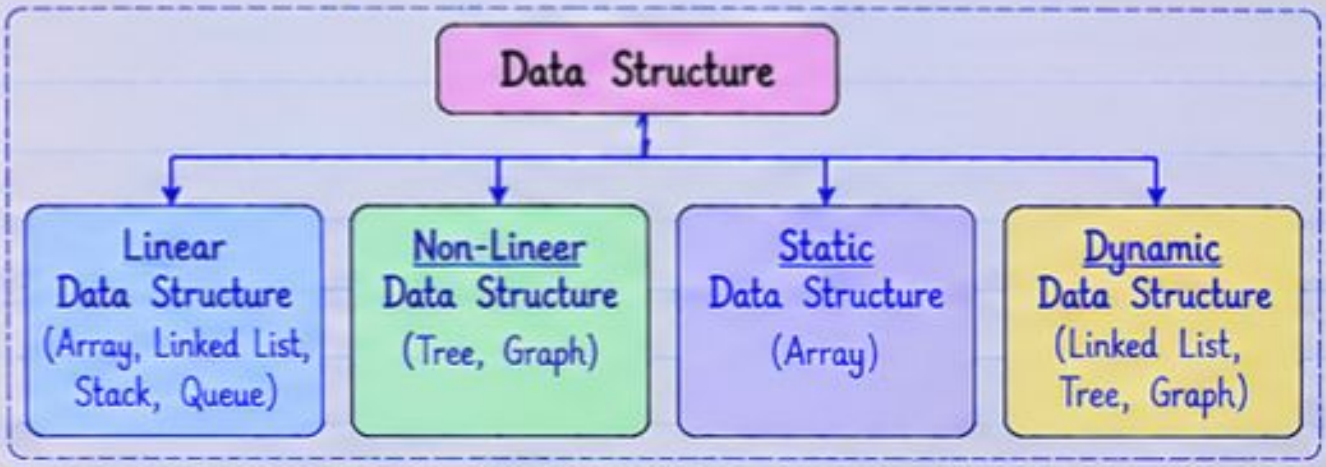
S. No.	Topic	Page No.
Part 1 : Introduction to DSA		
1	Cover Page	1
2	Table of Contents	2
3	Introduction to Data Structure and Algorithm	3
4	Algorithm Analysis and Complexity	4
Part 2 : Linear Data Structures		
5	Arrays	5
6	Searching Algorithms (Linear & Binary Search)	6
7	Sorting Algorithms - Part 1 (Bubble, Selection, Insertion)	7
8	Sorting Algorithms - Part 2 (Merge Sort, Quick Sort)	8
9	Linked List	9
10	Stack	10
11	Queue	11
Part 3 : Non-Linear Data Structures		
12	Introduction to Trees	12
13	Binary Search Tree (BST)	13
14	Heap and Priority Queue	14
15	Graphs	15
16	Graph Traversal (BFS and DFS)	16
17	Hashing and Hash Tables	17
Part 4 : Core Algorithms and Techniques		
18	Recursion	18
19	Divide and Conquer	19
20	Greedy Algorithms	20
21	Dynamic Programming	21
22	Backtracking	22
23	Two Pointers and Sliding Window	23
24	Prefix Sum and Binary Search Patterns	24
Part 5 : Advanced DSA and Problem Solving		
25	Trie and String Algorithms	25
26	Shortest Path and Advanced Graph Concepts	26
27	Union-Find and Minimum Spanning Tree	27
28	DSA Interview Patterns and Practice	28
29	Complexity Cheat Sheet and Revision	29
30	Practice Questions and Learning Roadmap	30

Roshani Kumari

1. Introduction to Data Structure and Algorithm

1.1 What is a Data Structure?

- A data structure is a way of organizing and storing data in a computer so that it can be used efficiently.
- It helps to manage large amounts of data and perform operations like insertion, deletion, searching and sorting easily.
- Examples: Array, Linked List, Stack, Queue, Tree, Graph, etc.



1.2 What is an Algorithm?

- An algorithm is a step-by-step procedure to solve a problem.
- It takes some input, processes it and gives the desired output.
- It must be clear, finite and effective.
- Example: Steps to find the largest number in an array.

Example Algorithm : Find Largest Number in an Array

1. Start.
2. Assume first element as largest.
3. Compare each element with largest.
4. If current element is greater, update largest.
5. Repeat until all elements are checked.
6. Return the largest number.

1.3 Importance of DSA

- Helps in writing efficient and optimized code.
- Improves problem-solving and logical thinking skills.
- Used in real-world applications like search engines, databases, operating systems, etc.
- Very important for technical interviews and placements.

2. Algorithm Analysis and Complexity

2.1 What is Algorithm Analysis?

- Algorithm analysis is the process of calculating the time and space required by an algorithm to solve a problem.
- It helps to compare different algorithms and choose the best one.
- It is independent of hardware and programming language.

2.2 Types of Complexity

Time Complexity	Space Complexity
• It calculates the time required by an algorithm to run. • It depends on the number of basic operations.	• It calculates the memory required by an algorithm. • It includes input space and auxiliary space (extra space).

2.3 Cases in Time Complexity

Best Case	Average Case	Worst Case
Minimum time required. Occurs for the most favourable input.	Expected time for most of the inputs.	Maximum time required. Occurs for the least favourable input.

2.4 Big-O Notation

- Big-O notation is used to represent the upper bound of time complexity.
- It shows how the running time of an algorithm increases with input size (n).
- It ignores constant factors and lower order terms.

Common Time Complexities

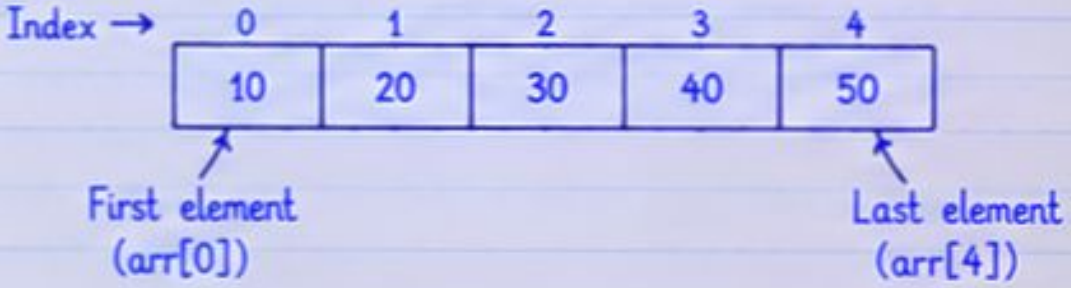
Notation	Name	Example
$O(1)$	Constant Time	Accessing an array element
$O(\log n)$	Logarithmic Time	Binary Search
$O(n)$	Linear Time	Linear Search
$O(n \log n)$	Linearithmic Time	Merge Sort, Quick Sort (avg)
$O(n^2)$	Quadratic Time	Bubble Sort, Selection Sort
$O(2^n)$	Exponential Time	Recursive Fibonacci (naive)

3. Arrays :

3.1 What is an Array?

- An array is a linear data structure that stores a fixed-size collection of elements of the same data type.
- Elements are stored in contiguous (adjacent) memory locations.
- Each element is accessed using an index.
- Indexing starts from 0.

3.2 Representation of Array



3.3 Types of Arrays

- One Dimensional Array (1D) - single row.
- Two Dimensional Array (2D) - rows and columns (matrix).
- Multi Dimensional Array - 3D, 4D, etc.

3.4 Basic Operations on Array

Traversal	Insertion	Deletion	Searching
Accessing each element one by one.	Adding a new element at a specific position.	Removing an element from a specific position.	Finding an element in the array.

3.5 Examples in Python

```
# 1D Array
arr = [10, 20, 30, 40, 50]
print(arr[0])   # 10
print(arr[3])   # 40
```

```
# Insertion (using list)
arr.append(60)
print(arr)    # [10, 20, 30, 40, 50, 60]
```

```
# Traversal
for i in range(len(arr)):
    print(arr[i], end=" ")
# Output: 10 20 30 40 50
```

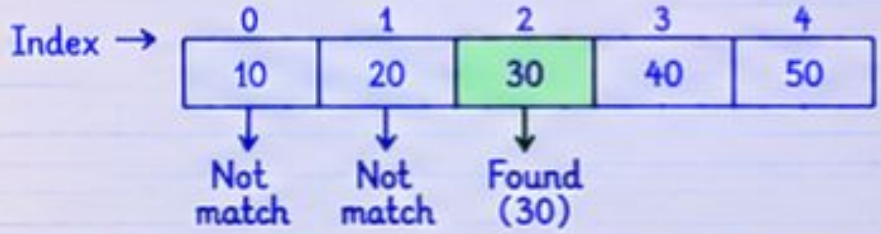
```
# Deletion (using list)
arr.remove(30)
print(arr)    # [10, 20, 40, 50, 60]
```

4. Searching Algorithms :-

4.1 Linear Search

- Linear search is a simple searching algorithm.
- It checks each element one by one until the desired element is found or the array ends.
- It works on both sorted and unsorted arrays.

Example : Search 30 in the array



Time Complexity:

Best Case	: O(1)
Average Case	: O(n)
Worst Case	: O(n)

Python Code : Linear Search

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i      # index found
    return -1             # not found
```

4.2 Binary Search

- Binary search is a faster searching algorithm.
- It works only on sorted arrays.
- It compares the middle element and decides whether to search in the left half or right half.
- It repeatedly reduces the search space by half.

Example : Search . 50 in the sorted array



Time Complexity:

Best Case	: O(1)
Average Case	: O(log n)
Worst Case	: O(log n)

Python Code : Binary Search

```
def binary_search(arr, key):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

5. Sorting Algorithms - Part 1

5.1 Introduction

- Sorting means arranging the elements of a list in a particular order (ascending or descending).
- Common sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort.
- Sorting helps in searching, data analysis and optimization.

5.2 Bubble Sort

- Repeatedly compare adjacent elements and swap if they are in wrong order.
- In each pass, the largest element moves to the end.

Example : Sort [5, 3, 4, 1] using Bubble Sort

Pass 1 : | 5 | 3 | 4 | 1 | → | 3 | 4 | 1 | 5 |
swap swap

Pass 2 : | 3 | 4 | 1 | 5 | → | 3 | 1 | 4 | 5 |

Pass 3 : | 3 | 1 | 4 | 5 | → | 1 | 3 | 4 | 5 |

5.3 Selection Sort

- Find the smallest element from the unsorted part and swap it with the first element.
- Repeat for the remaining elements.

Example : Sort [64, 25, 12, 22] using Selection Sort

Pass 1 :	12	25	64	22	(smallest 12)
Pass 2 :	12	22	64	25	(smallest 22)
Pass 3 :	12	22	25	64	(sorted)

5.4 Insertion Sort

- Take one element at a time and insert it into its correct position in the sorted part.
- Works like sorting cards in hand.

Example : Sort [5, 2, 4, 6, 1] using Insertion Sort

Step 1 : | 5 | 2 | 4 | 6 | 1 | → | 2 | 5 | 4 | 6 | 1 |

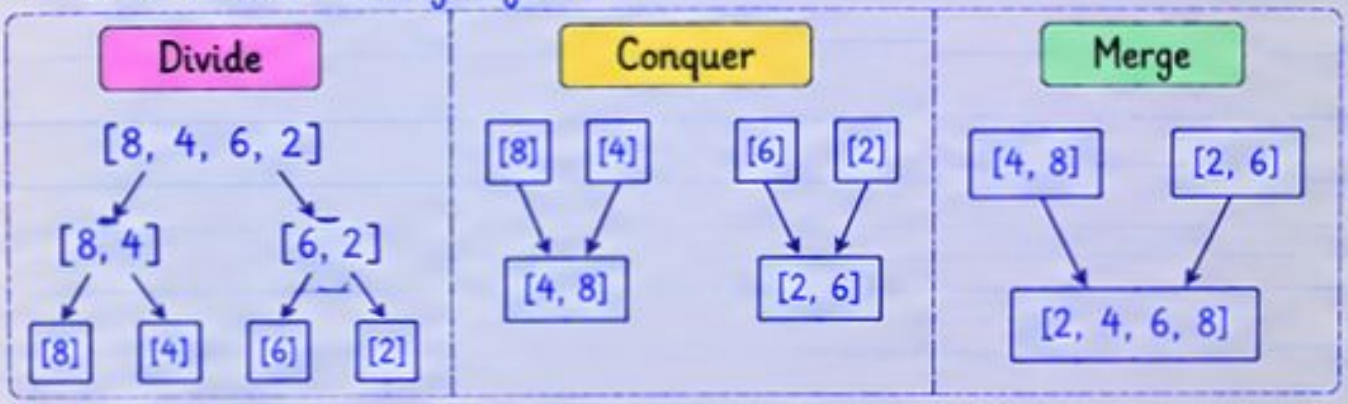
Step 2 : | 2 | 5 | 4 | 6 | 1 | → | 2 | 4 | 5 | 6 | 1 |

Step 3 : | 2 | 4 | 5 | 6 | 1 | → | 2 | 4 | 5 | 6 | → | 1 | 2 | 4 | 5 | 6 |

6. Sorting Algorithms - Part 2

6.1 Merge Sort

- Merge sort is a divide and conquer algorithm.
- It divides the array into two halves, sorts them recursively and then merges them.
- It is a stable sorting algorithm.



Python Code : Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

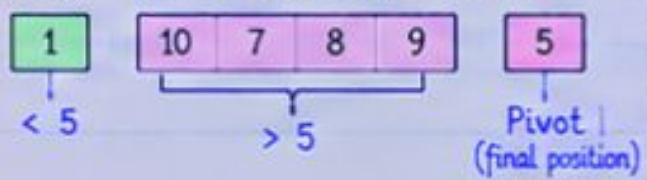
```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

6.2 Quick Sort

- Quick sort is also a divide and conquer algorithm.
- It selects a pivot element, partitions the array and recursively sorts the subarrays.
- It is not a stable sorting algorithm.

Partitioning Example

Array : [10, 7, 8, 9, 1, 5] Pivot = 5



Steps :

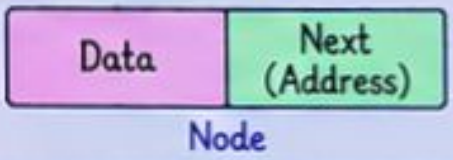
1. Choose a pivot (e.g. last element).
2. Partition the array (smaller elements on left, larger on right).
3. Recursively sort left and right subarrays.

7. Linked List

7.1 What is a Linked List?

- A linked list is a linear data structure where elements (called nodes) are stored in non-contiguous memory locations.
- Each node contains data and a reference (pointer) to the next node.
- It is dynamic in size (can grow or shrink during runtime).

7.2 Node Structure




```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

7.3 Types of Linked List

1. Singly Linked List



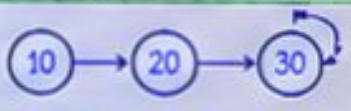
- Each node points to next node.
- Traversed in one direction.
- Simple and easy to implement.

2. Doubly Linked List



- Each node has two pointers (previous and next).
- Can be traversed in both directions.

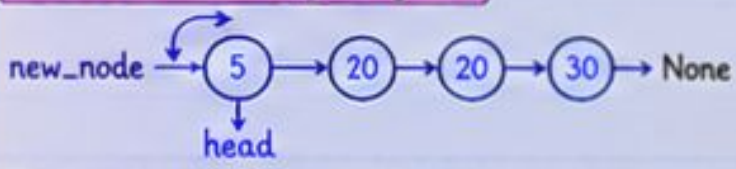
3. Circular Linked List



- Last node points to first node. No NULL at the end.
- Used in applications like round-robin scheduling.

7.4 Insertion in Singly Linked List

Insert at the Beginning :

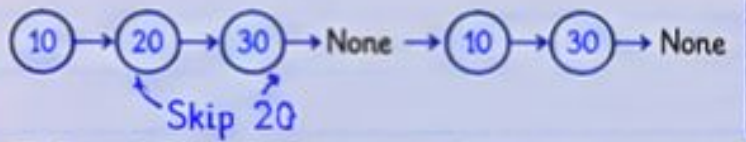


Python Code :

```
new_node = Node(5)
new_node.next = head
head = new_node
```

7.5 Deletion in Singly Linked List

Delete a Node (e.g. 20) :



Python Code :

```
temp = head
while temp.next and temp.next.data != 20:
    temp = temp.next
if temp.next:
    temp.next = temp.next.next
```

7.6 Advantages and Limitations

Advantages	Limitations
• Dynamic size.	• Random access is not possible.
• Efficient insertion and deletion.	• Extra memory required for pointers.
• No need for contiguous memory.	• More complex implementation.

8. Stack :

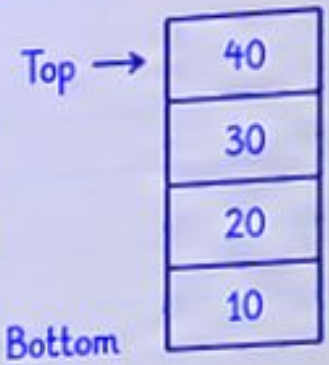
8.1 What is a Stack?

- A stack is a linear data structure that follows the LIFO (Last In First Out) principle.
- The last element inserted is the first one to be removed.
- Elements are added and removed from the top only.

8.2 Stack Operations

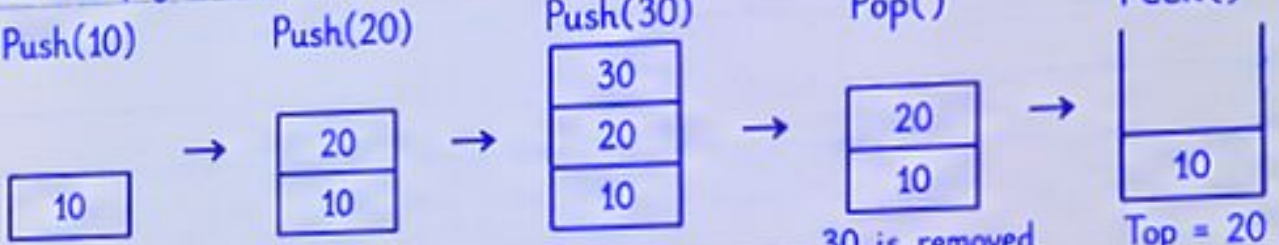
Operation	Description
Push()	Adds an element on the top.
Pop()	Removes the top element.
Peek()/Top()	Returns the top element without removing it.
isEmpty()	Checks if the stack is empty.
isFull()	Checks if the stack is full (in case of fixed size).

Stack Representation



8.3 Stack Operations Example

Initial : Empty Stack



8.4 Python Implementation using List

Python Code : Stack

```
stack = []
stack.append(10)      # push
stack.append(20)
stack.append(30)
print(stack)          # [10, 20, 30]
print(stack.pop())    # 30
print(stack[-1])      # top element -> 20
```

8.5 Applications of Stack

- Undo/Redo in text editor
- Browser history
- Function calls (recursion)
- Expression evaluation
- Backtracking problems
- Syntax checking in compilers

8.6 Overflow and Underflow

- Overflow: Occurs when we try to push an element into a full stack.
- Underflow: Occurs when we try to pop an element from an empty stack.

Roshani Kumari

9. Queue :

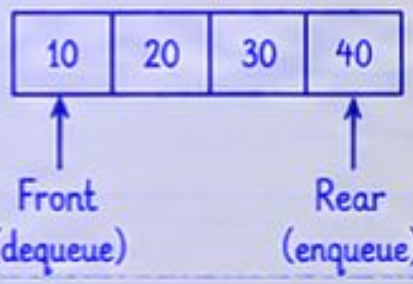
9.1 What is a Queue?

- A queue is a linear data structure that follows the FIFO (First In First Out) principle.
- The first element inserted is the first one to be removed.
- Elements are added at the rear and removed from the front.

9.2 Basic Operations

Operation	Description
Enqueue()	Adds an element at the rear.
Dequeue()	Removes an element from the front.
Front()	Returns the front element.
Rear()	Returns the last element.
isEmpty()	Checks if the queue is empty.
isFull()	Checks if the queue is full (in case of fixed size).

Queue Representation



9.3 Types of Queue

1. Simple Queue
- Basic queue with FIFO principle.
- Insertion at rear Deletion from front.

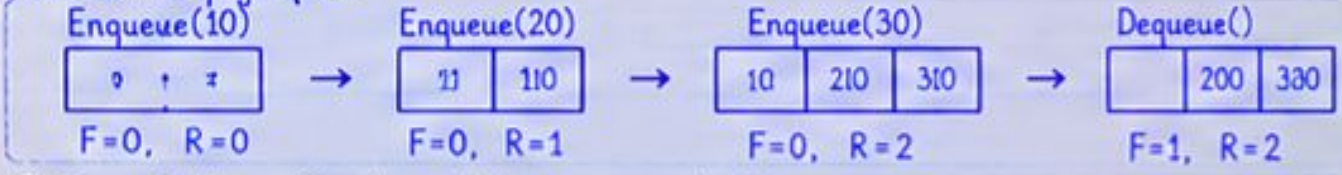
2. Circular Queue
- Last position is connected to first.
- Overcomes space wastage problem.

3. Deque (Double Ended)
- Insertion and deletion from both front and rear.
- Input restricted and output restricted.

4. Priority Queue
- Each element has a priority.
- Element with higher priority is removed first.

9.4 Example: Queue Operations

Initial : Empty Queue



9.5 Python Implementation using List

Python Code : Queue

```
from collections inport deque
q = deque()
q.append(10)          # enqueue
q.append(20)
print(q)              # deque([10, 20, 30])
print(q.popleft())    # 10 (dequeue)
print(q[0]            # front element -> 20
```

9.6 Applications of Queue

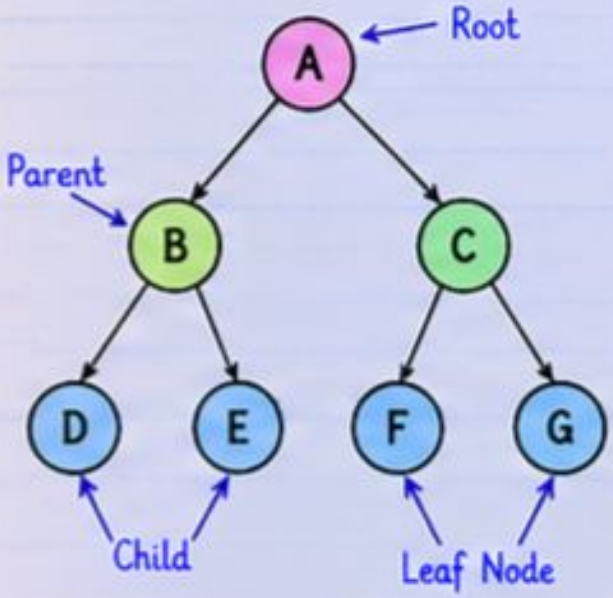
- CPU scheduling
- Printer job scheduling
- Message queues in systems
- Traffic management
- Breadth-First Search (Graph)
- Real-life example: people standing in a line

10. Introduction to Trees :

10.1 What is a Tree?

- A tree is a non-linear data structure that consists of nodes connected by edges.
- It has a hierarchical structure with a single root node.
- It is used to store data in a parent-child relationship.

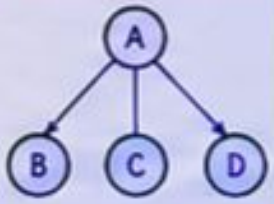
10.2 Basic Terminologies



Term	Meaning
Node	An element of a tree.
Root	Topmost node of the tree.
Edge	Connection between two nodes.
Parent	A node that has child nodes.
Child	A node directly connected to a parent node.
Leaf Node	A node with no child.
Sibling	Nodes with the same parent.
Level	Distance from the root node.
Height	The length of the longest path from root to a leaf.
Depth	Distance from the root to a node.

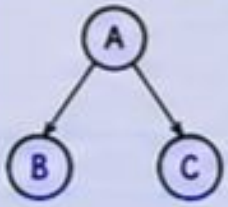
10.3 Types of Trees

1. General Tree



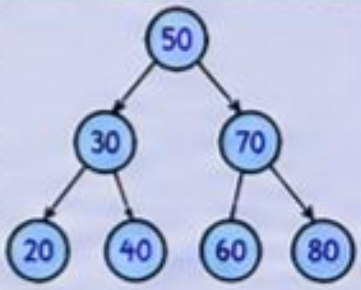
A node can have multiple children.

2. Binary Tree



Each node has at most two children.

3. Binary Search Tree



Left < Root < Right.

4. Other Trees

- AVL Tree
- Heap
- B-Tree
- Trie
- Expression Tree etc.

10.4 Real-World Applications

- File systems (directory structure)
- Databases (indexing using trees)
- Expression parsing (syntax trees)
- Hierarchical data (e.g., organization chart)
- Searching and sorting (BST)
- Used in compilers and networks

13. Binary Search Tree (BST) :

13.1 What is BST?

- A Binary Search Tree (BST) is a binary tree in which:
- Left subtree contains values smaller than the root.
- Right subtree contains values greater than the root.
- Both left and right subtrees are also BSTs.

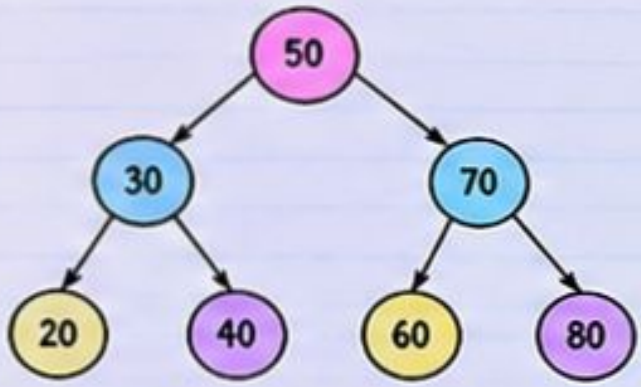
13.2 Properties

- Each node has at most two children.
- Left < Root < Right.
- No duplicate values (or handled in a specific way).
- Efficient search, insertion and deletion.

BST Condition

Left Subtree < Root
Right Subtree > Root

13.3 Example of a BST



Why it is BST?

- All values in left subtree are smaller than 50.
- All values in right subtree are greater than 50.
- Left and right subtrees are also BSTs.

13.4 Operations

1. Insertion

- Compare value with root.
- Go to left if smaller.
- Go to right if greater.
- Insert at correct position.

2. Searching

- Compare with root.
- Go left or right.
- Repeat until found or NULL.

3. Deletion

- Case 1: Node is leaf.
- Case 2: Node has one child.
- Case 3: Node has two children (replace with inorder successor).

13.5 Tree Traversal

Traversal	Order	Example (for above tree)
Inorder (Left-Root-Right)	Left → Root → Right	20 30 40 50 60 70 80
Preorder (Root-Left-Right)	Root → Left → Right	50 30 20 40 70 60 80
Postorder (Left-Right-Root)	Left → Right → Root	20 40 30 60 80 70 50

14. Heap and Priority Queue :

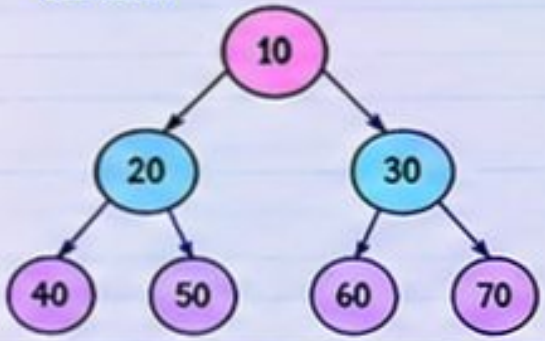
14.1 What is Heap?

- A heap is a complete binary tree that satisfies the heap property.
- It is usually implemented using an array.
- Two types: Min Heap and Max Heap.

14.2 Types of Heap

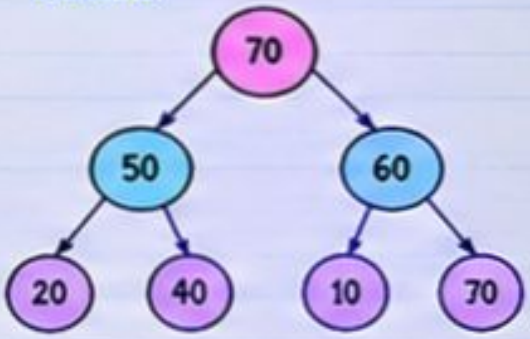
1. Min Heap

- Parent node is smaller than its children.
- Root contains the minimum element.



2. Max Heap

- Parent node is greater than its children.
- Root contains the maximum element.

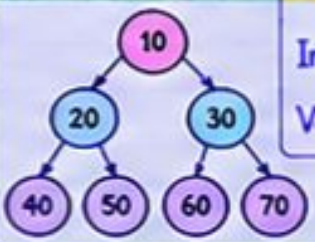


14.3 Heap Properties

- Complete binary tree (all levels filled except possibly last level, filled from left to right).
- Min Heap: parent ≤ children.
- Max Heap: parent ≥ children.
- For a node at index i (in array):
  - Left child = 2i + 1
  - Right child = 2i + 2
  - Parent = (i - 1) // 2

Array Representation (Min Heap)

Tree



Array

Index :	0	1	2	3	4	5	6
Value :	10	20	30	40	50	60	70

14.4 Operations

Operation	Description	Time Complexity
Insert	Add element at the end and heapify up	O(log n)
Remove Root	Remove min/max element and heapify down	O(log n)
Peek	Get the root element	O(1)
Build Heap	Create heap from array	O(n)

14.5 Priority Queue

- A priority queue is a data structure where each element has a priority.
- Elements with higher (or lower) priority are served first.
- It can be implemented using a heap.

Applications

- Task scheduling
- CPU scheduling
- Dijkstra's algorithm
- Event handling
- Print queue

Roshani Kumari

15. Graphs :

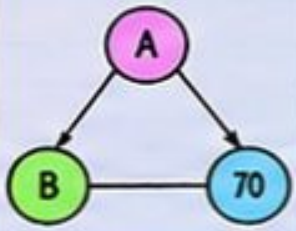
15.1 What is a Graph?

- A graph is a non-linear data structure that consists of a set of vertices (nodes) and a set of edges that connect the vertices.
- Graph is written as G = (V, E) where,
  - V = Set of vertices (nodes).
  - E = Set of edges (connections).

15.2 Types of Graphs

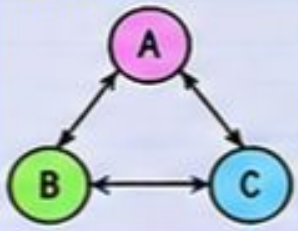
1. Undirected Graph

Edges have no direction.



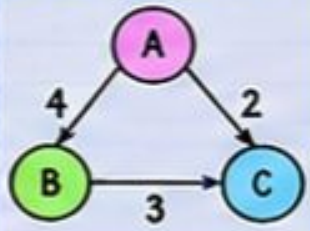
2. Directed Graph

Edges have direction (arrows).



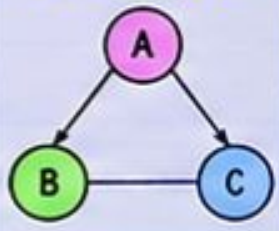
3. Weighted Graph

Edges have weights (cost or distance).



4. Unweighted Graph

All edges have same weight (usually 1).



15.3 Graph Representation

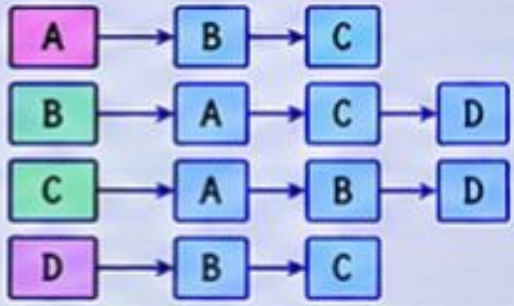
1. Adjacency Matrix

- A 2D array of size V × V.
- matrix[i][j] = 1 if edge exists.
- Good for dense graphs.

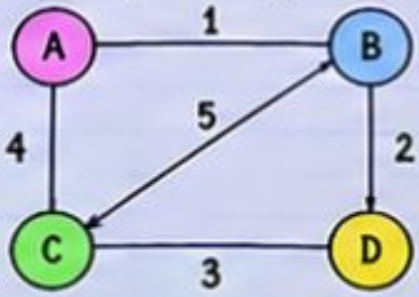
	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

2. Adjacency List

- Each vertex stores a list of its neighbours.
- Good for sparse graphs.



15.4 Example Graph



15.5 Applications

- Maps and navigation (shortest path)
- Social networks (friend connections)
- Computer networks
- Web pages and links (search engines)
- Transportation systems
- Recommendation systems

16. Graph Traversal : BFS and DFS

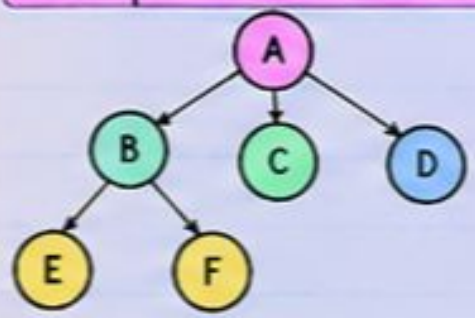
16.1 Introduction

- Graph traversal means visiting all the vertices (nodes) of a graph in a specific order.
- It is used to explore and process the nodes of a graph.
- Two main graph traversal algorithms are BFS (Breadth-First Search) and DFS (Depth-First Search).

16.2 Breadth-First Search (BFS)

- BFS visits nodes level by level.
- It uses a Queue (FIFO).
- It starts from a source node and first visits all its neighbours, then their neighbours, and so on.

Example : BFS Traversal



Traversal order (starting from A):

A → B → C → D → E → F

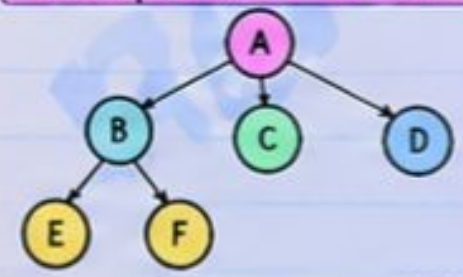
Queue operations (BFS)

Initial : [A]
Visit A : [B, C, D]
Visit B : [C, D, E, F]
Visit C : [D, E, F] ...

16.3 Depth-First Search (DFS)

- DFS goes as deep as possible along a path before backtracking.
- It uses a Stack (LIFO) or recursion.
- It starts from a source node and explores one branch fully before moving to another.

Example : DFS Traversal



Traversal order (starting from A):

A → B → E → F → C → D

Stack/Recursion (DFS)

Call Stack (recursive calls):

A → B → E → F → C → D

16.4 Comparison : BFS vs DFS

Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Traversal Order	Level by level	Go deep first
Use Case	Shortest path (unweighted graph)	Exploring all paths
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V)	O(V)

17. Hashing and Hash Tables

17.1 What is Hashing?

- Hashing is a technique to store and retrieve data quickly using a key.
- It uses a hash function to convert a key into an index (hash value).
- The data is stored in a hash table (array of buckets).
- It provides average case O(1) time complexity for search, insert and delete.

17.2 Hash Table Representation

Key → Hash Function → Hash Table (Index)

"apple" → h(key) → 3

0	—
1	banana
2	mango
3	apple
4	—
5	orange
6	—

17.3 Hash Function

- A hash function takes a key as input and returns an index.
- It should be fast and distribute keys evenly.
- Example : h(key) = key % size (for integer keys)
- For string keys, a different hash function is used.

17.4 Collision

- A collision occurs when two different keys get the same hash value.
- Since multiple keys can map to the same index, we need a collision handling technique.

Example of Collision

Hash table size = 10
h(key) = key % 10

h(22) = 22 % 10 = 2
h(32) = 32 % 10 = 2

Both keys map to index 2 (collision).

17.5 Collision Handling Techniques

1. Chaining (Separate Chaining)

- Each index stores a linked list of keys.
- If a collision occurs, the new key is added to the linked list.

Index 2 → 22 → 32 → 42

2. Open Addressing

When a collision occurs, we search for the next empty slot.
Common methods: Linear Probing, Quadratic Probing, Double Hashing.
Insert 22 → index 2
Insert 32 → index 3 (next empty)

17.6 Time Complexity

Operation	Average Case	Worst Case
Search	O(1)	O(n)
Insert	O(1)	O(n)
Delete	O(1)	O(n)

17.7 Applications

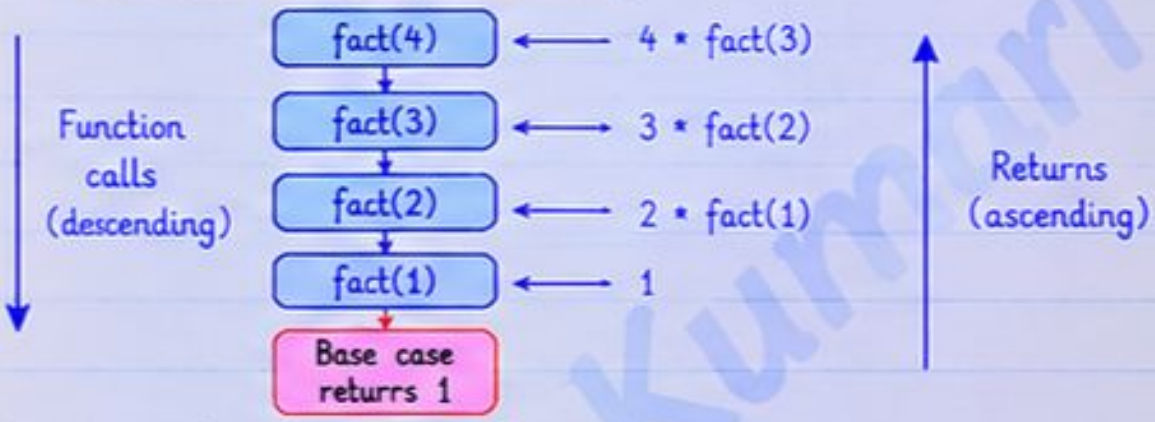
- Database indexing
- Caching (e.g., in web browsers)
- Symbol tables in compilers
- Storing unique elements (e.g., sets)
- Dictionary implementation (key-value pairs)

18. Recursion

18.1 What is Recursion?

- Recursion is a technique where a function calls itself to solve a smaller subproblem of the same problem.
- It is based on the idea of divide and conquer.
- Every recursive function has two parts:
  1. Base Case : stops the recursion.
  2. Recursive Case : calls the function again with a smaller input.

18.2 Recursion Flow (Call Stack)



18.3 Example 1 : Factorial

- Factorial of a number n is the product of all positive integers from 1 to n.
- Example : 5! = 5 × 4 × 3 × 2 × 1 = 120

Recursive Formula :

$n! = n \times (n-1)!$

Base case : $0! = 1$

Python Code :

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

18.4 Example 2 : Fibonacci Series

- Fibonacci series is a sequence where each term is the sum of the previous two terms.
- Example : 0, 1, 1, 2, 3, 5, 8, 13, ...

Recursive Formula :

$F(n) = F(n-1) + F(n-2)$

Base cases :
$F(0) = 0$, $F(1) = 1$

Python Code :

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1 :
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

18.5 Advantages and Limitations

Advantages	Limitations
Simple and easy to understand. Useful for problems like tree traversal, divide and conquer, backtracking.	Can be slower due to repeated calls. Uses extra memory (call stack). May cause stack overflow for large inputs.

19. Divide and Conquer :

19.1 What is Divide and Conquer?

- It is an algorithmic technique in which a problem is divided into smaller subproblems of the same type.
- Each subproblem is solved recursively.
- Finally, the solutions of subproblems are combined to get the final solution.
- It is useful for problems that can be broken into independent subproblems.

19.2 Steps of Divide and Conquer

1. Divide
- Break the problem into smaller subproblems.

→

2. Conquer

Solve the subproblems recursively.

→

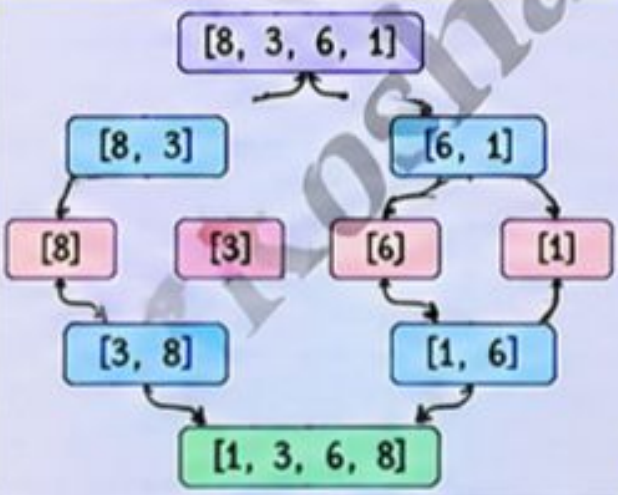
3. Combine

Merge the solutions of subproblems to get final answer.

19.3 Example: Merge Sort

- Divide the array into two halves.
- Sort both halves recursively.
- Merge the two sorted halves.

Merge Sort Process



Python Code (Merge Sort)

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

# merge() is a function to
# combine two sorted arrays
```

19.4 Example: Binary Search

- Divide the sorted array into two halves.
- Check the middle element.
- If found, return the index.
- Otherwise, search in left or right half.

When to Use?
- When a problem can be divided into smaller similar subproblems. Common in sorting, searching and many advanced algorithms.
- Examples: Merge Sort, Quick Sort, Binary Search.

Roshani Kumari

20. Greedy Algorithms :

20.1 What is Greedy Algorithm?

- Greedy algorithm makes the best choice at each step (locally optimal choice) with the hope of finding the global optimal solution.
- It does not reconsider past choices.
- It works well for problems that have optimal substructure and greedy choice property.

20.2 Characteristics

	When it gives Optimal Solution?
• Makes locally best choice at each step. • Does not backtrack. • Simple and efficient. • Gives optimal solution for some problems.	When the problem has: 1. Greedy choice property 2. Optimal substructure

20.3 Example: Activity Selection Problem

- Select the maximum number of activities that can be performed without overlapping.
- At each step, choose the activity with the earliest finishing time.

Activity	A1	A2	A3	A4	A5
Start Time	1	3	0	5	8
Finish Time	2	5	6	7	9

Selected Activities

A1 (1, 2)
A2 (3, 4)
A4 (5, 7)
A5 (8, 9)

20.4 Example: Coin Change (Greedy Approach)

- Given coins of certain denominations, find the minimum number of coins to make a target amount.
- Always pick the largest coin ≤ remaining amount.

Coin denominations

Step	Remaining Amount	Coin Chosen
1	37	20
2	17	10
3	7	5
4	2	2
Total coins = 4 (20 + 10 + 5 + 2)		

Python Code (Coin Change)

```
def coin_change(coins, amount):
    coins.sort(reverse=Tree)
    count = 0
    for coin in coins:
        while amount >= coin:
            amount -= coin
            count += 1
    return moant
```

20.5 Limitations

- Does not always give optimal solution.
- Fails for problems like 0/1 Knapasck.
- Need to check if the problem has greedy choice property.

Roshani Kumari

21. Dynamic Programming (DP) :

21.1 What is Dynamic Programming?

- Dynamic Programming is a technique to solve problems by breaking them into overlapping subproblems and storing their solutions.
- It avoids repeated calculations and improves efficiency.
- It is used when the problem has:
  1. Overlapping subproblems
  2. Optimal substructure

21.2 Memoization vs Tabulation

Memoization (Top-Down)	Tabulation (Bottom-Up)
• Solves problem using recursion. • Stores results in a memo table. • Computes only required subproblems. • Uses recursion + extra space (similar to call stack).	• Solves problem using iteration. • Stores results in a table (usually array). • Computes all subproblems in order. • No recursion, so no call stack.

21.3 Example: Fibonacci Series

1, 1, 2, 3, 5, 8, 13, ...
Each term is the sum of previous two terms.

Recursive (Simple)

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) +
           fib(n-2)
```

Time Complexity: O(2^n)
(Recomputes subproblems)

Memoization (Top-Down)

```
def fib(n, memo={}):
    if n <= 1:
        return n
    if n in memo:
        return memo[n]
    memo[n] = fib(n-1, memo) +
              fib(n-2), memo)
    return memo[n]
```

Time Complexity: O(n)
Space Complexity: O(n)

Tabulation (Bottom-Up)

```
def fib(n):
    if n <= 1:
        return n
    dp = [0] * (n+1)
    dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1]+dp[i-2]
    return dp[n]
```

Time Complexity: O(n)
Space Complexity: O(n)

21.4 DP Table for n = 6

n	0	1	2	3	4	5	6
fib(n)	0	1	1	2	3	5	8

Applications of DP

- Fibonacci series
- 0/1 Knapsack
- Longest Common Subsequence (LCS)
- Edit Distance
- Matrix Chain Multiplication
- Shortest Path Problems (e.g., Floyd-Warshall)

Roshani Kumari

22. Backtracking :

22.1 What is Backtracking?

- Backtracking is an algorithmic technique to solve problems by trying different possibilities.
- It builds the solution step by step (choose), explores further, and if it is not valid, it undoes the choice (backtrack).
- It is used for problems with multiple possible solutions.
- It is similar to brute force but with early pruning.

22.2 Steps in Backtracking

1. Choose	2. Explore	3. Check	4. Undo (Backtrack)
Pick a possible option.	Go deeper with that option (recursively).	If solution is valid or not.	If not valid, remove the choice and try next option.

22.3 Example: Maze Solving

- Find a path from start (S) to destination (D) in a maze.
- Move in 4 directions: up, down, left, right.

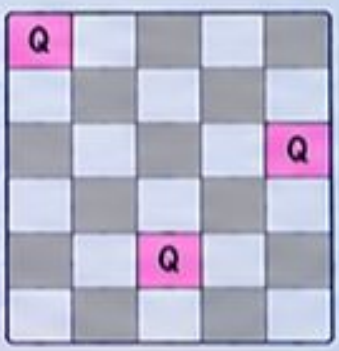


Python Code (Maze Backtracking)

```
def solveMaze(r, c):
    if r < 0 or c < 0 or r >= n
      or c >= n or maze[r][c] == 0:
        return False
    if r == n-1 and c == n-1:
        return True
    maze[r][c] = -1  # mark visited
    if (solveMaze(r+1, c) or
        solveMaze(r-1, c) or
        solveMaze(r, c+1) or
        solveMaze(r, c-1)):
        return True
    maze[r][c] = 1   # backtrack
    return False
```

22.4 Example: N-Queens Problem

- Place N queens on an N × N chessboard such that no two queens attack each other.
- Use backtracking to try column positions row by row.



22.5 Applications

- Maze solving
- N-Queens problem
- Sudoku solver
- Graph coloring
- Subset sum problem
- Combinational problems

Roshani Kumari

23. Two Pointers and Sliding Window :

23.1 Two Pointers Technique

- Two pointers technique uses two indices to solve array or string problems.
- Useful in sorted arrays or to find pairs, subarrays, etc.
- It helps to reduce time complexity from $O(n^2)$ to $O(n)$.

23.2 Example: Pair Sum in Sorted Array

Problem: Given a sorted array and a target sum, find two numbers whose sum is equal to target.

Array: [1, 2, 4, 6, 8, 10]

Target = 12

Steps:

- left = 0 (1)
- right = 5 (10)
- 1 + 10 = 11 < 12 → move left right
- 1 + 10 = 11 < 12 → move right
- 2 + 10 = 12 → found (2, 10)

Python Code

```
def twoSum(arr, target):
    left, right = 0, len(arr)-1
    while left < right:
        total = arr[left] + arr[right]
        if total == target:
            return (arr[left], arr[right])
        elif total < target:
            left += 1
        else:
            right -= 1
    return None
```

23.3 Sliding Window Technique

- Sliding window is used to find a subarray or substring that satisfies a condition.
- We maintain a window (range) and slide it across the array/string.
- There are two types

1. Fixed Size Window

Window size is constant (e.g., size k).

2. Variable Size Window

Window size changes based on the condition (e.g., sum ≤ target).

23.4 Example: Maximum Sum of Subarray of Size k

Problem: Find the maximum sum of any subarray of size k.

Array: [2, 1, 5, 1, 3, 2] **k = 3**

Window:

2	1	5	1	3	2

Sum = 8 (2+1+5) | Sum = 7 (1+5+1) | Sum = 9 (5+1+3) | Sum = 6 (1+3+2)

Python Code

```
def maxSum(arr, k):
    n = len(arr)
    curr_sum = sum(arr[:k])
    max_sum = curr_sum
    for i in range(k, n):
        curr_sum += arr[i] - arr[i-k]
    max_sum = max(max_sum, curr_sum)
    return max_sum
```

23.5 Applications

- Finding pair in sorted array
- Subarray problems (sum, length, etc.)
- String problems (longest substring, anagram, etc.)

Roshani Kumari

24. Prefix Sum and Binary Search Patterns

24.1 Prefix Sum

- Prefix Sum is an array where each element at index i stores the sum of all elements from index 0 to i.
- It helps to find the sum of a subarray in O(1) time after preprocessing.
- Useful for range sum queries.

24.2 Example: Prefix Sum Array

Array: [2, 4, 6, 8, 10]

Prefix Sum: [2, 6, 12, 20, 30]

Index (i)	0	1	2	3	4
Array[i]	2	4	6	8	10
Prefix[i]	2	6	12	20	30

Range Sum Query

Sum from index l to r = prefix[r] - prefix[l-1] (if l > 0)

Example: Sum from 1 to 3 = 20 - 2 = 18 (4 + 6 + 8)

24.3 Python Code (Prefix Sum)

```
arr = [2, 4, 6, 8, 10]
n = len(arr)
prefix = [0] * n
prefix[0] = arr[0]
for i in range(1, n):
    prefix[i] = prefix[i-1] + arr[i]

# Range sum from l to r
def rangeSum(l, r):
    if l == 0:
        return prefix[r]
    else:
        return prefix[r] - prefix[l-1]
```

Applications

- Range sum queries
- Subarray sum problems
- Counting subarrays with given sum
- Used in competitive programming

24.4 Binary Search Patterns

- Binary search is used to find an element or boundary in a sorted array.
- It works by repeatedly dividing the search space into half.
- Common patterns:

Pattern	Description	Example
1. Basic Binary Search	Find a target element	Search 7 in sorted array
2. First Occurrence	Find first position of target	First index of 5
3. Last Occurrence	Find last position of target	Last index of 5
4. Lower Bound	First element ≥ target	Smallest number ≥ x
5. Upper Bound	First element > target	Smallest number > x

Python Code (Basic Binary Search)

```
def binarySearch(arr, target):
    left, right = 0, len(arr)-1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

24.5 Common Mistakes

- Not handling edge cases (empty array, single element).
- Wrong mid calculation or loop condition.
- Confusing first and last occurrence.

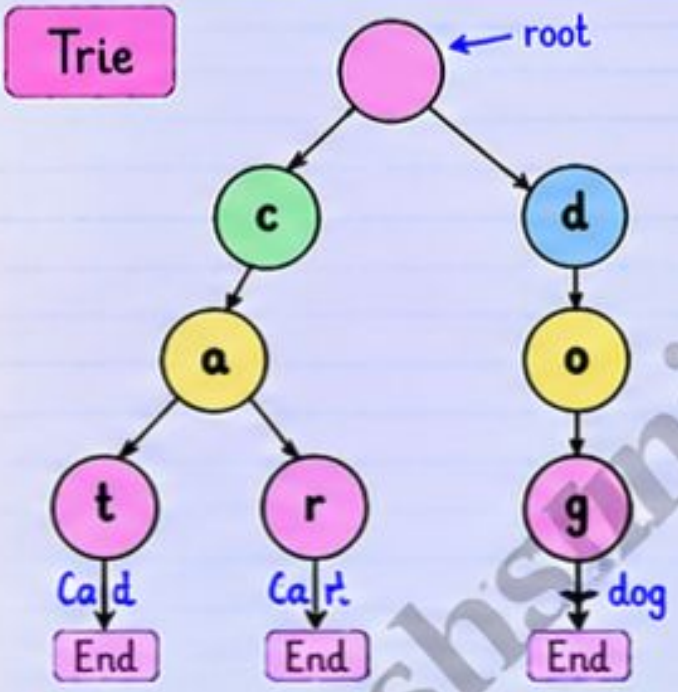
Roshani Kumari

25. Trie and String Algorithms :

25.1 What is Trie?

- Trie (pronounced as "try") is a tree like data structure used to store strings.
- Each node represents a single character.
- It is also known as Prefix Tree.
- It is mainly used for fast prefix search.

25.2 Example: Insertion of Words (cat, car, dog)



Operations in Trie

- Insert a word
- Search a word
- Check if a prefix exists
- Delete a word

Applications

- Autocomplete (search suggestions)
- Spell checkers
- Dictionary (store words)
- IP routing
- Phone contact search

25.3 Python Code (Insert and Search)

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False
class Trie:
    def __init__(self):
        self.root = TrieNode()
    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
                node = node.children[ch]
        node.end = True
```

```
def search(self, word):
    node = self.root
    for ch in word:
        if ch not in node.children:
            return False
        node = node.children[ch]
    return node.end
```

25.4 Other String Algorithms

- String Reversal: s[::-1]
- Palindrome Check: s == s[::-1]
- String Matching (KMP, Rabin-Karp)
- Anagram Check (sorted or frequency count)
- Longest Common Prefix

Roshani Kumari

26. Shortest Path and Advanced Graph Concepts

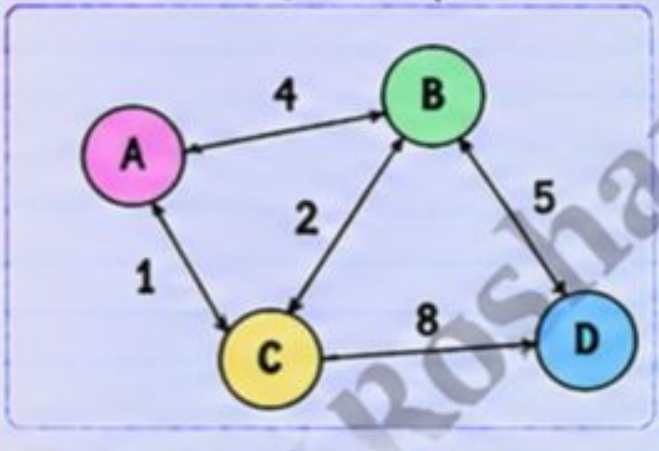
26.1 Shortest Path Problem

- In a weighted graph, shortest path means finding the path with minimum total weight between two vertices.
- It is used in maps, navigation systems, networks, etc.
- Different algorithms are used depending on the graph type.

26.2 Dijkstra's Algorithm (for non-negative weights)

- It finds the shortest path from a single source to all other vertices in a graph with non-negative edge weights.
- It always chooses the nearest unvisited vertex.
- Time Complexity: $O(V + E) \log V)$ using a priority queue.

26.3 Example Graph



Dijkstra from A

Vertex	Shortest Distance
A	0
B	4
C	1
D	6 (A → B → D)

26.4 Python Code (Dijkstra using Priority Queue)

```
import heapq
def dijkstra(graph, start):
    dist = {node: float("inf") for node in graph}
    dist[start] = 0
    pq = [(0, start)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

26.5 BFS for Unweighted Shortest Path

- In an unweighted graph, BFS can be used to find the shortest path.
- It gives minimum number of edges from source to destination.
- Time Complexity: $O(V + E)$

26.6 Other Concepts

- Bellman-Ford Algorithm (handles negative weights)
- Floyd-Warshall Algorithm (all pairs shortest paths)
- A* Algorithm (used in AI and pathfinding)

Roshani Kumari

27. Union-Find and Minimum Spanning Tree

27.1 Disjoint Set Union (DSU)

- Disjoint Set Union (DSU) is a data structure to keep track of a set of disjoint (non-overlapping) sets.
- It supports two main operations:
  1. Find(x): find the representative (parent) of the set.
  2. Union(x, y): merge two sets.
- It is used in cycle detection, Kruskal's algorithm, etc.

27.2 DSU with Path Compression and Union by Rank

```
parent = [i for i in range(n)]
rank = [0]*n
def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])   # path compression
    return parent[x]

def union(x, y):
    rx, ry = find(x), find(y)
    if rx == ry:
        return
    if rank[rx] < rank[ry]:
        parent[rx] = ry
    elif rank[rx] > rank[ry]:
        parent[ry] = rx
    else:
        parent[ry] = rx
        rank[rx] += 1
```

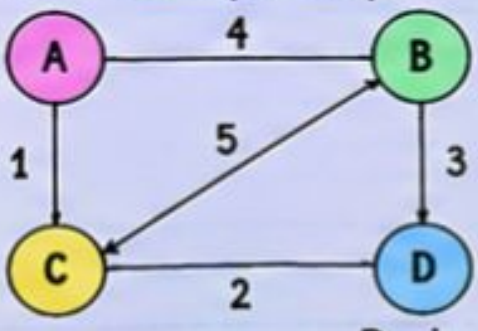
Applications of DSU

- Cycle detection in graph
- Kruskal's minimum spanning tree
- Network connectivity
- Dynamic connectivity problems

27.3 Minimum Spanning Tree (MST)

- A Minimum Spanning Tree (MST) is a subgraph of a connected weighted undirected graph that connects all vertices with minimum total weight.
- It has (V - 1) edges and no cycles.

27.4 Example Graph



27.5 Kruskal's Algorithm (Steps)

1. Sort all edges by weight.
2. Start with an empty MST.
3. Pick the smallest edge.
4. If it does not form a cycle (use DSU), add it to MST.
5. Repeat until (V - 1) edges are selected.

Time Complexity: O(E logE)

Roshani Kumari

28. DSA Interview Patterns and Practice

28.1 Common DSA Interview Patterns

- Interview questions follow some common patterns.
- These patterns help to identify the right approach quickly.
- Use data structures, algorithms and complexity analysis.
- Always try brute force first, then optimize.

28.2 Important Problem Patterns

1. Arrays

- Two pointers
- Sliding window
- Prefix sum
- Kadane's algorithm
- Sorting

2. Strings

- Palindrome
- Anagram
- Frequency count
- Two pointers
- String matching

3. Linked List

- Reversal
- Cycle detection
- Find middle
- Merge two lists
- Remove Nth node

4. Stack & Queue

- Valid parentheses
- Next greater element
- Sliding window max
- Queue simulation
- Monotonic stack

5. Trees

- Traversals
- Lowest common ancestor
- Diameter
- Serialize/Deserialize
- Tree to array problems

6. Graphs

- BFS/DFS
- Shortest path
- Cycle detection
- Topological sort
- Connected components

28.3 Brute Force vs Optimized Approach (Example: Two Sum)

Brute Force ($O(n^2)$)

```
def twoSum(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i+1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

Optimized using Hash Map ($O(n)$)

```
def twoSum(nums, target):
    map = {}
    for i, num in enumerate(nums):
        if target - num in map:
            return [map[target - num], i]
        map[num] = i
    return []
```

28.4 How to Explain a Solution in Interview

1. Understand the problem (restate in own words).
2. Discuss brute force approach.
3. Optimize and explain the better approach.
4. Analyze time and space complexity.
5. Write clean code and test with examples.
6. Mention edge cases and improvements.

28.5 Tips for Interviews

- Practice coding regularly.
- Know time and space complexity.
- Write clean and modular code.
- Think aloud while solving.
- Ask questions if unclear.
- Solve problems on arrays, strings, trees, graphs and DP.

Roshani Kumari

29. Complexity Cheat Sheet and Revision

29.1 Time Complexity of Common Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)

29.2 Time and Space Complexity of Data Structures

Data Structure	Access	Search	Insertion	Deletion	Space
Array	O(1)	O(n)	O(n)	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)	O(1)	O(n)
Stack	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	O(n)	O(n)	O(1)	O(1)	O(n)
Binary Search Tree	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Heap (Priority Queue)	O(1)	O(n)	O(log n)	O(log n)	O(n)
Hash Table	O(1)	O(1)	O(1)	O(1)	O(n)
Trie	O(m)	O(m)	O(m)	O(m)	O(n)
Graph (Adj List)	-	O(V + E)	O(1)	O(1)	O(V + E)

29.3 Important Definitions

- Time Complexity: Time taken by an algorithm as a function of input size.
- Space Complexity: Memory used by an algorithm.
- Big-O Notation: Represents upper bound of time/space.
- Best Case: Minimum time (most favourable input).
- Average Case: Expected time for random input.
- Worst Case: Maximum time (most unfavourable input).
- Stable Sorting: Relative order of equal elements is maintained.
- In-place Algorithm: Uses constant extra space.

29.4 Quick Revision Diagram



Roshani Kumari

30. Practice Questions and Learning Roadmap

30.1 Practice Questions

1. Arrays (Easy)

1. Find the largest element in an array.
2. Find the second largest element.
3. Rotate an array by k positions.
4. Move all zeroes to the end.
5. Find the missing number (1 to n).

2. Searching (Easy)

1. Implement linear search.
2. Implement binary search.
3. Find first and last occurrence of an element in a sorted array.

3. Sorting (Medium)

1. Implement bubble sort.
2. Implement selection sort.
3. Implement merge sort.
4. Implement quick sort.
5. Sort an array of 0s, 1s and 2s.

4. Stack (Medium)

1. Check for valid parenthess.
2. Implement a stack using array.
3. Implement a stack using linked list.
4. Find next greater element.

5. Tree (Medium)

1. Inorder, preorder and postorder traversal of a binary tree.
2. Find the height of a tree.
3. Check if a tree is a valid BST.
4. Find lowest common ancestor (LCA).

6. Graph (Advanced)

1. Implement BFS and DFS.
2. Find shortest path in an unweighted graph.
3. Detect cycle in a graph.
4. Find number of connected components.

30.2 Learning Roadmap (Beginner → Advanced)

1. Basics (1-2 weeks)
- Learn basics of programming (Python)
- Understand time and space complexity
- Solve easy array and string problems.

→

2. Core Data Structures (3-6 weeks)
- Arrays, Linked List
- Stack, Queue
- Trees, Graphs
- Hashing, Trie, Heap
- Solve medium level problems.

→

3. Core Algorithms (6-10 weeks)
- Searching & Sorting
- Recursion
- Divide & Conquer
- Greedy Algorithms
- Dynamic Programming
- Backtracking

→

4. Advanced & Practice (10+ weeks)
- Graph Algorithms
- Advanced DS
- Solve mock tests
- Practice on platforms (LeetCode, GFG, etc.)
- Revise regularly.

30.3 Final Checklist and Key Takeaways

- Understand and implement all major data structures.
- Learn when to use which data structure.
- Practice coding problems regularly.
- Analyze time and space complexity.
- Be comfortable with recursion and DP.
- Solve problems on online platforms.
- Revise important patterns and formulas.
- Practice previous year interview questions.
- Build small projects using DSA concepts.
- Keep learning and stay consistent.

Roshani Kumari